

AI Paper Archive — Project Context

Product brief + discussion notes · 18 Sep 2026

Part 1 — Original product brief

Product idea

Build an extremely simple AI-powered paper/document archive that lets users digitize physical mail and paperwork and eventually throw away originals when appropriate.

The core onboarding goal is:

| *Sign in with Google → grant access → start scanning.*

There should be essentially no setup. Users should not need to configure folders, databases, categories, naming conventions, API keys, etc.

The application should work on iOS and Android, initially as a PWA/web app rather than separate native apps.

Core user flow

1. User signs in with their Google account.
2. App creates/uses its own document area in the user's Google Drive.
3. User taps Scan.
4. Camera opens.
5. User photographs one or more pages.
6. App performs document cleanup where possible: page detection, crop, perspective correction, contrast/readability improvements.
7. Pages are submitted for processing.
8. Google Document AI OCR extracts canonical text/layout.
9. An LLM receives the OCR result and optionally the original document images to understand the document.
10. The LLM produces structured metadata.
11. Final PDF/image is stored in the user's Google Drive.
12. OCR text, metadata, search information and the Google Drive file ID are stored in Postgres.
13. User gets a simple result showing what was filed and whether action is required.

Example result:

固定資産税納税通知書

上越市 · 2026/04/03

¥128,400

Due: 2026/04/30

⚠ Action required: payment

📄 Keep original

[View document](#)

Main product screens

Keep the application extremely simple.

Scan — Large scan button optimized for quickly processing incoming physical mail. Support multi-page documents.

Recent / Inbox — Shows recently scanned documents, processing status, required actions and documents that need user review.

Search — Search all scanned documents using normal keywords or natural language. Examples:

- 固定資産税
- Tokyo Gas
- documents from 上越市
- Find that tax letter I received a few months ago
- Show electricity bills from last year

OCR + AI architecture

Use Google Document AI Enterprise OCR as the canonical OCR layer. Its responsibility is: *“What text/layout is actually present on this document?”*

Then use an LLM for: *“What does this document mean?”*

Example OCR:

上越市
令和8年度
固定資産税・都市計画税 納税通知書
納期限 令和8年9月30日
納付額 128,400円

The LLM converts this into structured data:

```
{
  "document_type": "tax_notice",
  "title": "固定資産税納税通知書",
  "issuer": "上越市",
  "document_date": "2026-09-01",
  "due_date": "2026-09-30",
  "amount": 128400,
  "currency": "JPY",
  "action_required": true,
  "action": "Pay by 2026-09-30",
  "categories": ["tax", "property"],
  "summary": "Property and city planning tax notice",
  "retention": "keep",
  "tags": ["tax", "property", "2026"]
}
```

Store both the original OCR text and the AI interpretation. Do not make the LLM output the only representation of the document.

Physical-document retention

Help users decide whether the physical document can be discarded. Possible statuses:

-  Digital copy likely sufficient
-  Keep temporarily
-  Keep original
-  Unsure — user review required

Be conservative. The system should not guarantee that an original has no legal significance unless that can genuinely be established.

Actions

The larger product opportunity is not merely OCR or document storage. The application should understand what the physical mail wants the user to do.

Tax notice

¥34,800

Due September 30
⚠ Payment required
[Add reminder]

School letter

School trip: October 14–16
Permission form due September 25
[Add to calendar] [Remind me]

Potential extracted actions: payment deadlines, appointments, renewals, application deadlines, forms requiring signatures, events, contracts requiring response, cancellation deadlines.

Storage philosophy

The actual documents remain in the user's Google Drive:

```
Google Drive
├── Paper Archive
│   ├── 2026
│   │   └── 09
│   │       ├── 2026-09-18_上越市_固定資産税.pdf
│   │       └── 2026-09-18_東京電力_請求書.pdf
```

Folders are primarily for human convenience. The database is the real filing/indexing system.

Trust proposition: *“Your original documents stay in your Google Drive.”*

Avoid requiring unrestricted access to the user's entire Drive. Request the minimum Google OAuth permissions necessary.

Database

Use Neon/Postgres. Do not use Google Sheets as the production database.

Potential document model:

```
documents
  id
  user_id
  drive_file_id
  filename
  ocr_text
  ocr_provider
  ocr_confidence
  title
  document_type
  issuer
  document_date
  summary
  action_required
  action_type
  action_date
  retention_status
  retention_reason
  extracted_data JSONB
  created_at
  updated_at
```

Use JSONB for document-specific extracted fields because bills, tax documents, contracts, school letters, receipts, etc. have different schemas.

Potentially use pgvector later for semantic search. Use normal Postgres full-text search for straightforward OCR keyword searches so every search does not require an LLM call.

Proposed infrastructure

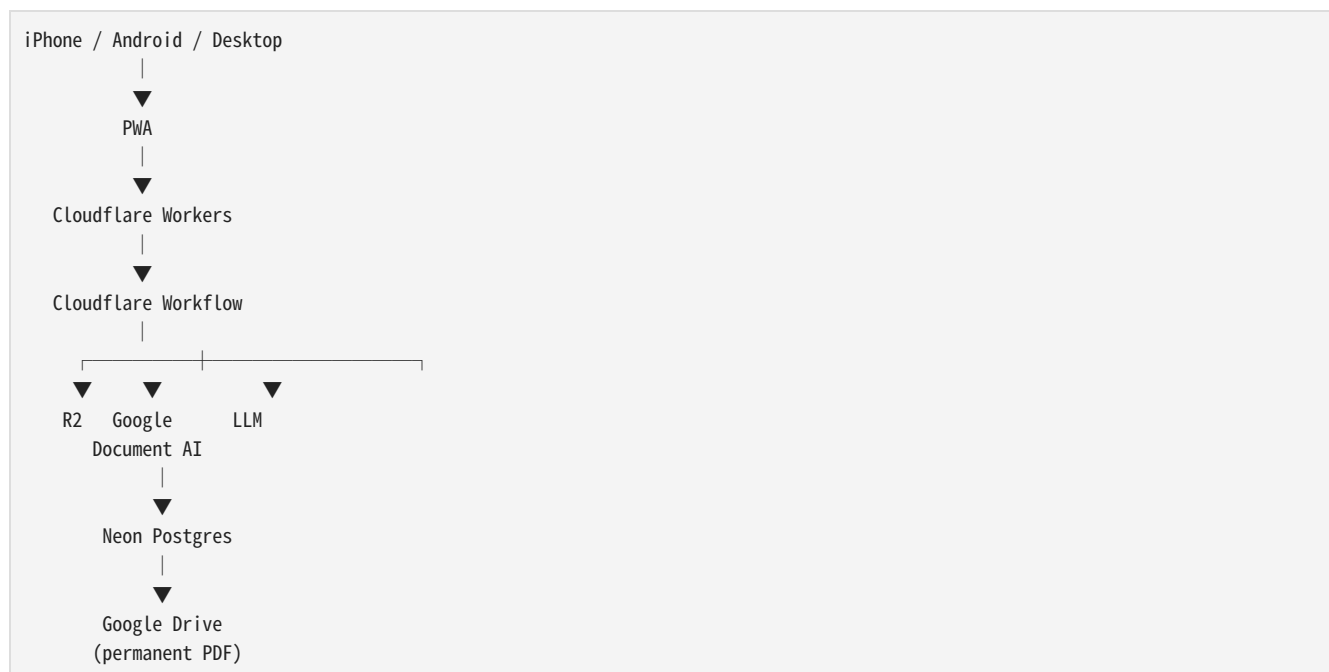
Layer	Choice	Notes
Frontend	React/Next.js PWA	One app for iPhone, Android, desktop. Capacitor wrap or native apps later if camera limits bite.
Backend	Cloudflare Workers	APIs, auth coordination, app logic
Background processing	Cloudflare Workflows / Queues	Async, durable, retryable stages
Temporary storage	Cloudflare R2	Uploaded images during processing; delete once in Drive
Permanent storage	User's Google Drive	
Database	Neon Postgres	
OCR	Google Document AI Enterprise OCR	
AI understanding	LLM (see Part 2)	
Search	Postgres FTS + metadata → later pgvector/embeddings	

Processing pipeline:

Upload → OCR → AI extraction → Validate → Generate final document
→ Upload to Google Drive → Index in Postgres → Complete

Individual stages should be retryable because Document AI, the LLM or Google Drive may temporarily fail or rate-limit.

Overall architecture:



MVP philosophy

Do not over-optimize AI/OCR costs initially. Start with: OCR every document → AI-process every document. Document AI OCR is inexpensive enough that routing/fallback logic is not worthwhile during the MVP.

Collect a real test corpus, especially Japanese documents: municipal notices, utility bills, tax documents, receipts, school letters, contracts, vertical Japanese text, mixed Japanese/English, poor photographs, folded documents, handwriting. Then measure extraction accuracy and optimize based on actual failures.

Product principle

The product should not feel like a document-management system. The experience should feel like:

| *Open mail → scan → throw into appropriate physical pile → forget about it.*

Later the user should be able to ask “Where’s that letter about my property tax?” and immediately retrieve the original document.

The long-term concept is broader than mail: **an AI interface to all of the user’s physical paperwork.**

Part 2 — Discussion notes and decisions (18 Sep 2026)

Model choice: Claude instead of OpenAI

- The “what does this document mean?” step is model-agnostic: text + images in, JSON out. Swapping OpenAI for Claude is a one-API-call change in the Workflow step; nothing else in the architecture cares.
- Claude handles Japanese well, accepts images directly (pass page photos alongside the Document AI OCR text), and structured JSON output works fine.
- Keep OCR on Document AI as the canonical layer; do not use an LLM as the OCR source of truth.
- “Astra” was raised as an alternative. If that meant Google’s Project Astra, it is a real-time assistant concept rather than an extraction API; the practical Google-side equivalent is Gemini via Vertex AI (also feasible, and keeps OCR + extraction + Drive on one vendor).
- Because raw OCR is stored separately, the model choice is not lock-in. A/B Claude vs Gemini vs OpenAI on the Japanese test corpus and pick by accuracy and cost.

MCP: add it, but do not replace extraction with it

Decision: keep structured extraction at ingest AND expose the archive to assistants via an MCP server.

Why MCP is a strong addition:

- Natural-language querying (“where’s the property tax letter?”, “what bills are due this month?”, “add a reminder”) is exactly what an assistant with tools does well. You get a chat interface for free.
- Makes the product a data layer rather than another app people must open — fits “scan and forget”.

Why extraction at ingest still matters:

- If only raw OCR is archived, every “what needs action?” query means the assistant re-reading and re-interpreting dozens of documents on demand: slow, expensive, inconsistent between calls, and the Inbox/action-required view cannot exist.
- Extract once, store structured metadata; MCP becomes a third consumer of the index alongside the PWA and search.

Suggested shape:

- **Keep:** scan → OCR → structured extraction → Drive + Postgres.
- **Trim:** the search UI can be minimal since MCP handles natural-language querying.
- **Add:** an MCP server over the Postgres index. Tools: `search_documents`, `get_document`, `list_actions`, `set_retention_decision`, `get_drive_link`.

Caveat: MCP as the *primary* interface assumes users already live in Claude or similar. For a consumer mail app most won’t, so the PWA scan flow and a basic Inbox still carry the product; MCP is the power-user path and the long-term “AI interface to all your paperwork” story.

Weekend build scope

A working end-to-end prototype (“scan → filed → ask Claude about it”) is realistic in a weekend if scoped hard. The full brief is not.

Fits in a weekend

- Google sign-in + creating a “Paper Archive” folder with the `drive.file` scope
- Photo upload from a web page (`<input type="file" capture="environment">`) → R2
- Document AI OCR call + Claude extraction to the JSON schema

- Neon table with the fields above, Postgres full-text search
- A crude Inbox list and a search box
- A small MCP server with `search_documents` and `get_document` (a few hours)

Defer

- Cloudflare Workflows for durable retries — a plain Worker running steps inline is fine for a demo; add Workflows after seeing real failures.
- Image cleanup (page detection, perspective correction) — skip; Document AI copes with mediocre photos. Native-quality scanning is a week of work.
- Multi-page PDF assembly — start with one image = one document.
- Retention logic and calendar/reminder actions — emit the fields, show as text, do not wire actions yet.
- Google OAuth verification for Drive scopes — multi-week process, irrelevant for a weekend but it is the gate before real users.

Risk: Google Cloud project setup, Document AI processor enablement, OAuth consent screen and service-account plumbing can burn half a Saturday. Do these Friday night.

Market landscape

The niche has become crowded in the last year, mostly with small indie apps, many built around German bureaucracy:

- **Paperwork** (iOS) — scan/import letters, auto-organize, highlight what matters, deadline reminders. German-bureaucracy focus, on-device storage, no account.
- **dokado** (iOS) — scans mail, imports PDFs/emails, extracts due dates, payments and addresses into task/info cards.
- **Papeer** — most ambitious: extracts tasks/deadlines/contracts/forms/appointments, builds a “memory” of your bureaucracy, executes follow-up actions with approval, can send physical letters back out.
- **Filex AI** — closest to the “AI interface to all paperwork” framing: scans attach to the person/property/organization they belong to; due/renewal/expiry dates detected with reminders.
- **Paperspell, AutoPA, MailScan AI, DigiMail** — similar scan → summarize → surface actions/dates apps.
- Incumbents: Adobe Scan, Microsoft Lens, Google Drive scanner (capture + OCR only, no meaning); Paperless-ngx for self-hosters.

Where this idea still differs

1. **Japan** — everything above is Western-centric. Japanese municipal notices, vertical text, 和暦 dates, and “納税通知書 → pay by X” patterns are a real gap.
2. **Your Drive, not our cloud** — most competitors own the storage; “originals stay in your Google Drive” is an unusual trust story.
3. **MCP / assistant-native** — none are exposed as a tool to Claude or other assistants.
4. **Discard guidance** — the retention status (digital copy sufficient vs keep original) is not something the others emphasize.

Action: try Papeer and Filex AI before the weekend to see what they got right and wrong.

Open items

- Confirm what “Astra” referred to.
- Decide MCP server hosting (same Cloudflare Worker vs separate).
- Define the retention-status rules and how conservative the defaults should be.
- Start collecting the Japanese test corpus.